What the firmware image says about the DMM6500's Lua API

`docs/UNDOCUMENTED.md` holds behaviour measured on the instrument, and every claim in it has been to the bench. This file is the other half: what can be read out of the **firmware image** — names, constants, values and structure that no Keithley document mentions. The two have different evidence bars on purpose, and mixing them would wreck the older file's.

The image is the only complete census available. On the instrument `next(display)` returns zero keys: the namespace is metatable-backed, so a name that has not been guessed cannot be discovered from a running DMM6500 at all.

Status	Meaning
`IMAGE`	Read out of a firmware table, where the name and its number are in the same record.
`CODE`	Read from the firmware's instructions or decompiled C.
`CONFIRMED`	`IMAGE` or `CODE`, and then checked on this DMM6500 at 1.7.17a.
`ABSENT`	In the image, and **not** present on this model — the pool is shared across Keithley's line.
`HAZARD`	Known to crash, wedge or saturate something. Stated with what to do instead.

Nothing in the calibration or adjustment namespaces is listed, probed or described here. Exclusion is token-wise, so `display.scalex`, `display.scaley` and `localnode` stay in — they merely contain the letters.

The machine the API is talking to

Two identical 400 MHz PowerPC processors sit on the CPU board with an FPGA that drives the display and touchscreen; one has 128 MB of DDR2, the other 256 MB. The analog front end has its own ARM Cortex-M4.

The firmware bundle names all of it. `versions.txt` inside `ki_DMM6500_v1_7_18a.zip`:

Component	Version	What it is
`6518`	v1.7.18a	the main board — `main.zx`, the application CPU
`afpgaalgo_1.sea`, `afpgadata_2.sed`	v0.36.0a	the analog FPGA
`amicro_3.zx`	v0.10.0a	the analog micro, the Cortex-M4
`6520` / `6521` / `6522`	v0.12.0a / v0.9.0a / v0.6.0a	the GPIB, TSP-Link and RS-232 comm cards, each with its own FPGA pair

Correction. An earlier revision of this file called the 128 MB processor "the raster graphics engine" and said every display call was a message to a different processor. That was wrong. `CODE`: the start-up function at `0x00146FCC` creates the instrument's tasks and then, with no role switch, calls `c2c_text_channel_init(9, 0x1977, 0xFB775, "DAQ Display", 'F', 'T')`. That function (`0x000A7394`) registers channel 9 as the "Display C2C Client" (string `0x00545380`, materialised at `0x000A7498`) and then creates the task "PEG Main" (string `0x00545394`, materialised at `0x000A7708`; 32

KB stack, priority 0xB4). It has exactly two call sites, `0x001479C4` and `0x00147AE8`, with byte-identical arguments. The widget tree, the graph's binning and autoscale, its tick engine and the line and marker rasterising are all in `main.x` and run in that task, on the same core as the Lua interpreter. Channel 9 is an in-process channel: the channel objects carry a local-or-remote flag at `+0x441`, and the local path calls the registered handler directly.

What is not established is where the finished pixels go. PEG draws through a screen object (`DAT_00eb3ec4`) whose driver is unread, and `c2c_text_channel_init` installs an interrupt handler for a device at `0x90000C00` and reads a 64-bit time base at `0x90000C20` — that `0x90000000` window is where frame-buffer scan-out and touch would live ("PEG Touch Controller", `0x00545060`). So a second processor is still doing display work, at the frame-buffer level rather than the widget level. Which processor has which role, and what `logic.zx` configures, is unread.

What survives is the part that matters for cost: a display call is still a message — to a task, not a processor — so the cost is per *call*, not per pixel. One `display.graph.add` hands over a buffer name and the receiving task does the rest; `display.setpixel` writes RAM directly and pays a Lua-to-C call per pixel.

The display task gets the core only when the Lua task blocks or yields. `CONFIRMED` on 1.7.17a: with the same paint pending in both cases, a synchronous `graph.removeall` cost 0.24 ms after an 80 ms `delay()` and 22 ms after an 80 ms busy spin on `os.clock()` — the spin left the display task exactly as far behind as no gap at all. So a Lua loop that never blocks starves it, and its work for a frame is paid either inside a synchronous call or inside a deliberate yield, never for free. The display queue is finite: `q_post_msg` at `0x0024104C` retries once after 100 ms on a full queue, then drops the message, and posts event `2351` for the first ten drops only (`CODE`). This bench has hung the instrument by flooding the display, with `1706` "Message dropped from display queue" in the log before the panic (`CONFIRMED`, `MEASUREMENTS.md`); which queue posts `1706` is unread. Outrunning the display task shows up as silent loss, not back-pressure, which is why this project holds its own display updates between 1 and 4 Hz.

Getting at the image

Nothing is encrypted. `file(1)` calls the decoded result an "OpenPGP Secret Key", which is a magic-byte coincidence.

```
ki_DMM6500_v1_7_18a.zip
  └ main.zx            itself a zip
      └ main.x         Motorola S-record, hex ASCII  (27.5 MB)
          └ decode     10,710,276 bytes in 11 runs, 0 bad lines -- the application CPU
  └ logic.zx           a zip holding logic.x, also S-records -- a Lattice FPGA bitstream
  └ rom.zip            holds afpgaalgo_1.sea and friends -- not yet examined
```

logic.zx is a Lattice bitstream, not Xilinx

`logic.x` decodes to exactly 1 MB at flash `0xFE000000` — one contiguous run, no gaps — of which `0xFE000000`-`0xFE091000` is populated and the rest is erased. It holds **no CPU instructions at all**: zero `mflr`, zero `blr`, zero ARM or MIPS return idioms, and 44 % of it is `0xFFFFFFFF`.

The layout is a 4-byte big-endian payload length, `0xFF` padding, then the bitstream:

```
FE000000  00 09 08 4F                      591,951 -- the exact payload length
FE000004  ff ... ff                        341 bytes of padding
FE000159  bd b3 ff ff ff ff 3b 00 00 00    0xBDB3 is the Lattice preamble
FE000167  e2 00 00 00 41 11 10 43          0xE2 verify-id, then IDCODE 0x41111043
```

`IMAGE`. **`0xBDB3` is Lattice Semiconductor's bitstream preamble.** Xilinx would begin `0xAA995566`, which does not occur anywhere in the file in any bit or byte order — I checked the plain image, the byte-swapped form, and the whole image bit-reversed, since a configuration PROM image is commonly stored bit-reversed. The IDCODE's low 12 bits, `0x043`, are Lattice's JTAG manufacturer field; the part field is `0x1111` and the version 4. There is exactly one preamble, so one device. A 65-byte autocorrelation peak in the payload fits frame-based configuration data.

The file carries no ASCII part name, design name or date — a Lattice `.bit` normally opens with a comment block naming the device, and this one has been stripped to padding.

`tools/` has nothing for this; the scripts live outside the repo because an extracted firmware archive is untrusted data. `/tmp/ghsc/srflat.py` flattens the S-records at their true addresses, which `desrec.py` did not do — it concatenated them, so every offset past the first gap was wrong.

The image runs from DDR at zero, not from flash

The S-record addresses are flash: eleven runs from `0xFF200000` to `0xFFF75EAC`, with the PowerPC reset vector at `0xFFF00100`. **The code does not run there.** 37,512 of the 39,898 addresses its `addis` / `addi` pairs build are in `0x00xxxxxx`: the image is copied into DDR and linked low.

The mapping was recovered by alignment rather than assumption. For a candidate delta, score how often a materialised address lands exactly on the first byte of a NUL-terminated string:

delta	lands on a string start
`0xFF200000`	**13.8 %**
`0xFF20A258`	2.8 %
`0xFF1FE96C`	2.8 %
four others	2.8 - 3.2 %

So `link = flash − 0xFF200000`, the image is linked at `0x00000000`, and an importer told the flash base resolves nothing. The four large runs are contiguous in link space, so `main.x` is the **application** CPU alone; whatever runs on the second processor is not in it.

The image states this itself, which beats inferring it. It carries its own section table — a Green Hills `.secinfo` at flash `0xFFBE2300` — giving `.text` at RAM `0x4000`, `0x5217D0` long, with the flash copy exactly `0xFF200000` higher. Two independent routes to the same delta, one of them the linker's own record. `IMAGE`. The scoring table above is kept because it is the method that works on an image with no section table at all.

Loading it into Ghidra, after `srflat.py` has written the app region:

```
analyzeHeadless <proj> <name> -import main_app.bin \
    -processor PowerPC:BE:32:default -loader BinaryLoader -loader-baseAddr 0x0
```

Ghidra 12.2 ships no macOS decompiler native. It builds from the bundled source in about a minute: `make -j10 ghidra_opt` in `Ghidra/Features/Decompiler/src/decompile/cpp`, then copy `ghidra_opt` to `Ghidra/Features/Decompiler/os/mac_x86_64/decompile`. No clone and no gradle needed.

How the API is registered, and why the pool over-reports it

Constants come from static tables of 8-byte records, `{ const char *name; int32_t value; }`, NULL-terminated and grouped by family. The names are stored fully qualified — `display.KEY_HELP`, `dmm.SLOPE_RISING` — so a table can be found by its shape alone, and a row cannot pair a name with somebody else's number. 742 of them are recovered this way across 23 namespaces: `dmm` 293, `display` 130, `buffer` 84, `trigger` 65, `channel` 31, `scan`, `ki`, `serial`, `analogtrigger`, `format`, `localnode`, `file`, `filter`, `math`, `tspnet`, `coupling`, `lan`, `rel`, `fan`, `threshold` and three singletons.

Functions are not a table. Each one is built at startup by a sequence of method calls on a descriptor object — set the name, append several strings, set a double — inside one large registrar per namespace. `display`'s is at link `0x002CC0B4`; the three-instruction stub at `0x002CC0A8` in front of it is a tail call, and an auto-analyser that believes the first call never returns will truncate the function there and show you two lines of C. No handler address appears in those blocks, so what is being built is command **metadata** and dispatch happens later by name.

The string pool is shared across Keithley's product line, so its names are not this model's API. Measured: `dump`, `capture`, `scalex`, `getpixel`, `drawstyle`, `generate`, `getobject`, `removeall`, `select`, `cursor`, `add`, `markersize`, `scaletrace`, `getchecksum`, `autolocal` and `press` are all in the `display` pool and all `nil` on a DMM6500 at 1.7.17a. A name from the image is a candidate; a `type()` lookup on the instrument is the answer. That lookup is also the safe way to ask, since it is a table read and never a call.

A constant is userdata wrapping its number, and `tonumber` gets the number out. Measured:

```
type(display.OBJ_CIRCLE)      userdata
tostring(display.OBJ_CIRCLE)  "display.OBJ_CIRCLE"
tonumber(display.OBJ_CIRCLE)  4
display.OBJ_CIRCLE == 4       false
```

So the integers in this file are both the firmware's internal ids and the values Lua can obtain. The comparison is false because Lua 5.0 only consults `__eq` when both operands have the same type — compare with `tonumber(x) == 4`, never `x == 4`.

This interpreter is a modified Lua 5.0.2. The metatable on these objects carries `__tostring`, `__tonumber`, `__eq`, `__add`, `__sub`, `__mul`, `__div`, `__unm` and `__concat`. Stock 5.0.2 has no `__tonumber` metamethod at all, so Keithley patched the core; and the arithmetic metamethods mean a constant can be added and concatenated directly, which is worth knowing before writing `tonumber()` everywhere.

A dotted string in the image is not an API name. The 10.7 MB holds about a thousand of them, and the residue includes source filenames (`interruptcontroller.c`, `serial_iodevice.c`), lower-case attribute paths used only when the instrument generates a configuration script (`filter.enable`, `rel.level`), and substrings sitting inside help text. `buffer.FILL_ONCE` appears at an unaligned address with no reference

anywhere in the image — it is a literal the instrument prints into a generated script, while the real constant is registered separately under its short name. So a dotted-name count is a starting point for a census and never the census itself.

The full catalog is `docs/firmware-constants.tsv`, with a `kind` column, because two different things share these tables. A final component in capitals is a constant — `display.KEY_HELP`. A lower-case one is an **attribute path** — `analogtrigger.edge.level` — whose number is a default or an offset rather than anything Lua reads. And 73 names appear in more than one table with different numbers, which means at least one of those tables is not a constant registry: every row of such a name is marked `DUP` so that nothing quotes one as a fact. **No `display.*` or `ki.*` constant is among them**, so everything in this file comes from a single table.

---

display: what exists and is undocumented

`CONFIRMED` by `type()` on 1.7.17a. None of these appears in either reference manual, the user manual, `DMM6500DisplayDoc.md` or `Display API Command Reference.md`.

Name	Kind	Note
`display.setfill`	function	takes a `FILL_*` direction
`display.setcell`	function	
`display.setfont`	function	the fourteen font constants; see `UNDOCUMENTED.md` for their measured heights
`display.setpixel`	function	`HAZARD`: measured too slow to draw with. Use `display.graph`
`display.setposition`	function	
`display.setthickness`	function	
`display.setminmax`	function	
`display.setabbrv`	function	
`display.setpopupmenu`	function	
`display.setevent`	function	takes an OR of the `EVENT_*` flags below
`display.setkeyenable`	function	measured in `UNDOCUMENTED.md`: disables a key, no getter
`display.flush`	function	not yet characterised — the one candidate for batching draws
`display.graph`	table	a sub-namespace, below
`display.activebuffer`	table	
`display.readingformat`	userdata	a property; reads back `display.FORMAT_PREFIX`
`display.lightstate`	userdata	a property; reads back `display.STATE_LCD_25` — the backlight
`display.watchchannels`	string	a property; empty on this instrument

display.graph

`CONFIRMED`. The graph widget is the fast drawing path: one call hands a buffer name to the display task, which keeps its own copy of the readings and draws from that. The mechanism, every member and the measured costs are under "The graph" below.

Confirmed function	Confirmed `nil`
`graph.scalex`, `graph.scaley`, `graph.scaletrace`	`graph.setminmax`, `graph.generate`
`graph.drawstyle`, `graph.markersize`, `graph.markerlastcolor`	`graph.getobject`, `graph.xscale`, `graph.yscale`
`graph.select`, `graph.cursor`, `graph.add`, `graph.removeall`	`graph.cursorlock`

Five of the ten — `add`, `drawstyle`, `scalex`, `scaley`, `removeall` — are also exercised every frame by a working application (`dmm6500-fft/tsp/spectrum_graph.tsp`). `notes/firmware/registry.json` marks eight of them `exists_on_dmm6500: false, "measured nil on 1.7.17a"`; that column is wrong for the whole namespace, because the lookups it records were made on `display`, where these names are nil, not on `display.graph`. The same file lists 16 `display.graph.set*` / `getvalue` / `create` rows and 44 `display.graph.SCREEN_*` / `COLOR_*` / `TEXT*` / `TIMER_FOREVER` constants. They are namespace-attribution artefacts: the registrar pushes one `graph` sub-table and then exactly ten methods (`0x002CCF78`-`0x002CD2FC`, `CODE`), and all 60 are `display.*` names.

`HAZARD`: **setting the graph's horizontal units to frequency crashes the instrument.** Time in microseconds works and is what to use. Measured on this bench before this document existed. What the code says about it, and the one experiment that would pin it, are under "The frequency crash" below.

The scaling constants live on `display`, not on `display.graph`. `IMAGE`: `XSCALE_SMART` 0, `XSCALE_LATEST` 1, `XSCALE_GROUP` 2, `XSCALE_ALL` 3, `XSCALE_AUTOBIN` 4, `XSCALE_SWIM` 5, `XSCALE_SHARED` 6, `XSCALE_OFF` 7; `YSCALE_SMART` 0, `YSCALE_PER_TRACE` = `YSCALE_ALL` 1, `YSCALE_PER_TRACE_ONCE` 2, `YSCALE_SWIM` 3, `YSCALE_SHARED` 4, `YSCALE_OFF` 5. `ELEMENT_DATA` 0, `ELEMENT_TIME` 1, and `ELEMENT_EXTRA` and `ELEMENT_SOURCE` are **both 2**, so passing one is indistinguishable from passing the other.

Object types, in full

`IMAGE`. The documented set is `OBJ_TEXT`, `OBJ_RECT`, `OBJ_LINE`, `OBJ_BUTTON`, `OBJ_BUTTON_MENU`, `OBJ_GRAPH`, `OBJ_SCREEN`, `OBJ_SCREEN_MENU`, `OBJ_SCREEN_HOME`, `OBJ_SWIPE` and the `OBJ_EDIT_*` family. The rest are not.

id	name	id	name
1	`OBJ_TEXT`	18	`OBJ_GRAPH`
2	`OBJ_RECT`	20	`OBJ_SCREEN`
3	`OBJ_LINE`	21	`OBJ_SCREEN_MENU`
4	**`OBJ_CIRCLE`**	22	`OBJ_SCREEN_HOME`
5	**`OBJ_IMAGE`**	25	`OBJ_SWIPE`
6	`OBJ_BUTTON`	**26**	**`OBJ_TIMER`**
7	`OBJ_BUTTON_MENU`	**30**	**`OBJ_PROGRESS_BAR`**
8	`OBJ_EDIT_NUMBER`	**31**	**`OBJ_POPUP_MENU_SCREEN`**
9	`OBJ_EDIT_OPTION`	32	`OBJ_EDIT_NOTIFY_EVENT`
10	`OBJ_EDIT_STRING`	33	`OBJ_EDIT_WAIT_EVENT`
11	`OBJ_EDIT_CHECK`	34	`OBJ_EDIT_BUFFER`

id	name	id	name
12	**`OBJ_EDIT_SLIDER`**	35	`OBJ_EDIT_CHANNEL`
		36	**`OBJ_LIST`**

`OBJ_CIRCLE`, `OBJ_IMAGE` and `OBJ_PROGRESS_BAR` are the interesting ones: a circle and an image are drawing primitives expressed as objects, which the display task draws from one message, and a progress bar is a widget this project currently fakes with a rectangle.

The firmware's own type-name table at `0x009B0FDC` goes past the enum (`CODE`): 13-17 are the input dialogs ("Input Option", "Input Number", "Input String", "Input Channel", "Prompt"), 19 "Screen", **23 "Graph Screen"**, 24 "Swipe", 37 "Window", 38 "Fast Text" and 39 "Static Label", none with a `display.OBJ_*` constant. 23 matters below: every `display.graph.*` function accepts an object of type 18 or 23 and nothing else (allow-list `{18, 23, 0}` at `0x009B11C8`, `IMAGE`).

Event flags are a bitmask

`IMAGE`, and the values say so — they are powers of two, so `display.setevent` takes an OR.

flag	value	flag	value
`EVENT_PRESS`	1	`EVENT_ENDAPP`	16
`EVENT_SCALE`	2	`EVENT_SHOW`	32
`EVENT_DRAG`	4	`EVENT_HIDE`	64
`EVENT_CURSOR`	8		

`EVENT_SHOW` and `EVENT_HIDE` are not mentioned anywhere outside the image.

Other display families worth knowing

`IMAGE`. Full list with values in `docs/firmware-constants.tsv`.

- `FONT_SMALL` 0, `FONT_MEDIUM` 1, `FONT_LARGE` 2, `FONT_HUGE` 3, `FONT_1`-`FONT_10` 4-13. Fourteen faces, and `UNDOCUMENTED.md` has their measured pixel heights — the ten numbered ones are not a size ladder.
- `JUST_LEFT` 0, `JUST_RIGHT` 1, `JUST_CENTER` 2 — text justification.
- `FILL_LEFT` 0, `FILL_RIGHT` 1, `FILL_UP` 2, `FILL_DOWN`.
- `STYLE_LINE` 0, `STYLE_MARKER` 1, `STYLE_BOTH` 2 — for `graph.drawstyle`.
- `STATE_ENABLE` 0, `STATE_DISABLE` 1, **`STATE_READONLY` 2**, `STATE_INVISIBLE` 3.
- `MARK_GREEN` 0, `MARK_YELLOW` 1, `MARK_RED` 2; `CURSOR_VERTICAL`, `CURSOR_HORIZONTAL`, `CURSOR_TRIGGER`.
- `BUTTON_OPTION1`-`BUTTON_OPTION10` 5-14; `BUTTONS_NONE` 0 through `BUTTONS_YESNOCANCEL` 5.
- `SFORMAT_ANY` 0 … `SFORMAT_FILE_NAME` 4, **`SFORMAT_IP_ADDRESS` 5**. This project shipped a two-field address editor because 21 name guesses missed `SFORMAT_IP_ADDRESS`; the image had it.
- `TYPE_NONE` 0, `TYPE_SCREEN` 1, `TYPE_BOTH` 3.
- `DUMP_NONE` 0, `DUMP_LIST` 1, `DUMP_BRIEF` 2, `DUMP_ALL` 3 — `ABSENT`, along with `display.dump`.

---

ki: the internal namespace, and the front-panel key table

`ki` is a table and `ki.display` is a table — `CONFIRMED`. It holds constants and nothing else: 29 names, every one of them an action code for a physical input. None is documented anywhere.

The expansion of `ki` is not in the image. "KeithleyInternal" appears nowhere in 10.7 MB; the only strings are `Keithley` and `keithley`. So the reading is plausible and unconfirmed.

Input	press	release
HOME	1	11
MENU	2	12
QUICKSET	3	13
HELP	**4**	**14**
ENTER	5	15
EXIT	6	16
FUNCTION	7	17
TRIGGER	8	18
FRONTREAR	9	19
OUTPUT	10	20
TOUCH	25	26
2TOUCH	33	34

Also `ACTION_MOVE_TOUCH` 32, `ACTION_MOVE_2TOUCH` 35 and `ACTION_VELOCITY` 36 for drag and swipe, and `ACTION_KNOB_LEFT` 21 with `ACTION_KNOB_RIGHT` 22 — **for a rotary knob the DMM6500 does not have.** The knob is on the DMM7510 and the SMUs. That is the clearest single piece of evidence that these tables are one shared codebase across Keithley's line, and the reason a name in the image is only ever a candidate until the instrument is asked.

The action codes and `display.KEY_*` are the same table. `display.KEY_HELP` is 4, which is `ki.display.ACTION_PRESS_HELP`; `KEY_TRIGGER` is 8, `KEY_FUNCTION` 7, `KEY_QUICKSET_APPS` 3. The documented `KEY_*` set names four of the twelve inputs, and `8` is also the `button=8` the instrument's own web panel posts to press TRIGGER, so the web interface drives this same table.

Using the HELP key: it cannot be done, and here is the evidence

No front-panel key except TRIGGER can reach a running Lua program on this firmware. That is a negative result, and it is worth as much as a feature, because the alternative is somebody spending a session looking for the route.

Four independent things close it off.

route	state
a trigger event from the key	`trigger.EVENT_DISPLAY` is the **only** display-sourced trigger event in the image. The complete list is `ANALOGTRIGGER`, `BLENDER`, `COMMAND`, `DIGIO`, `DISPLAY`, `EXTERNAL`, `LAN`, `NONE`, `NOTIFY`, `SCAN_ALARM_LIMIT`, `SCAN_CHANNEL_READY`, `SCAN_COMPLETE`, `SCAN_MEASURE_COMPLETE`, `TIMER`, `TSPLINK`. There is no per-key event and no `EVENT_KEY`.

route	state
`display.setkeyenable`	takes a key **away** from the firmware. It cannot report a press, and there is no `display.getkeyenable` — measured.
`display.generate`, `display.press`	the two functions that would synthesise an action: both `ABSENT` on this model.
the `EVENT_*` bitmask	`PRESS`, `SCALE`, `DRAG`, `CURSOR`, `ENDAPP`, `SHOW`, `HIDE` are all **object** events. None is a key.

`ki.display`'s 29 action codes are a constant table with no Lua-visible consumer: the only function that took them, `display.generate`, is not registered here. Outside `ki.*` the whole 10.7 MB image contains exactly five key-related names — the four `display.KEY_*` constants and `display.EVENT_PRESS`.

So what the HELP key can do from a script is exactly one thing: `display.setkeyenable(display.KEY_HELP, display.OFF)` stops it opening the firmware's help screen. Useful for an app that wants the glass to itself; useless as an input.

The way to get a help screen is a button, not the key. The firmware already has the widget: the system information screen puts up a modal **scrollable text window with its own scrollbar and an OK button**, and `display.prompt(enum disp_buttons, string) -> int` is the entry point that builds one. Whether it scrolls arbitrary length text, and what the length limit is, is the next thing to measure. An app's own on-screen Help button calling `display.prompt` would give the same result the key would have, and it works today.

Every display signature, because the signatures are data

The argument lists do not have to be recovered one handler at a time. **Every name in the `display` namespace resolves to the same C closure** — one function with four upvalues, where upvalue 3 points at a 12-byte descriptor holding `{method_index, in_spec, out_spec}`, and each spec carries the minimum argument count, the maximum, and an array of per-argument type words. So a signature is a table lookup, and all 60 came out mechanically.

`min..max` is enforced **before** the handler runs, and a violation posts SCPI `-109` for too few arguments or `-108` for too many. On this instrument that is a pop-up, so an argument count is worth getting right from the table rather than by experiment.

There is no per-name C handler anywhere in the firmware, which is why this is the only route. All 387 function, getter and setter registrations push the *same* `lua_CFunction`, with one exception. The trampoline reads its upvalues and calls a dispatcher that is not a switch: it **marshals** the call, writing the command id and module id as 16-bit fields into a buffer for a service task. A TSP call is a message, not a function call. That explains the measured cost — about 4 µs before anything happens — and it is the same architecture as the display: work is handed to another executor.

Three things establish that the reading is correct rather than plausible. `display.input.prompt` comes out as `(enum disp_buttons, string) -> enum` and `display.input.number` as `(string, int, double, double, double, [string])`, both exactly the documented signatures. The switch cases in the implementation match the recovered method indices 0 to 58 one for one. And event `1130` is posted from this path as `(name, min, max)`, which is the same `1130` this project already measured when `display.create` refused a `digits` value outside 4..9.

In the table below, **argument counts and types are `CODE`** — read out of the firmware's own metadata. The argument *names* are a reading of the implementation and are weaker. The **exists** column is `CONFIRMED` or `ABSENT`, measured by `type()` on 1.7.17a. `obj` is a handle from `display.create`.

Lua name	arguments (min..max)	returns	exists
`display.type`	—	enum `disp_type`	property, reads `TYPE_BOTH`
`display.flush`	(obj) 1..1	—	function
`display.setcell`	(obj, int, int, string) 4..4	—	function
`display.setfont`	(obj, enum `disp_font`) 2..2	—	function
`display.setfill`	(obj, int percent, enum `disp_fill`) 2..3	—	function
`display.setminmax`	(obj, double, double) 3..3	—	function
`display.setpixel`	(int, int, int, value, int) **4..5**	—	function
`display.setthickness`	(obj, int thickness) 2..2	—	function
`display.setposition`	(obj, int, int, int, int) 3..5	—	function
`display.setabbrv`	(obj, string × 10) 3..11	—	function
`display.setevent`	(obj, enum `disp_event`, value) 3..3	—	function
`display.setpopupmenu`	(obj, int, enum state, string) 3..4	—	function
`display.setkeyenable`	(enum `disp_key`, enum `disp_onoff`) 2..2	—	function
`display.readingformat`	(enum `disp_readingformat`) 1..1	mirror	property, `FORMAT_PREFIX`
`display.lightstate`	(enum `disp_light_state`) 1..1	mirror	property, `STATE_LCD_25`
`display.watchchannels`	(string) 1..1	mirror	property, empty
`display.activebuffer`	(buffer) 1..1	mirror	table
`display.format`	(double, string, int, int, enum `disp_rbelement`) 4..5	string	not probed
`display.graph.add`	(obj, buffer, enum elem, value, value, enum elem, string) 1..7	—	function
`display.graph.scalex`	(obj, enum `disp_xscalemethod`, enum axis, double, double) 1..5	(double, double)	function
`display.graph.scaley`	(obj, enum `disp_yscalemethod`, enum axis, double, double) 1..5	(double, double)	function
`display.graph.scaletrace`	(obj, enum `disp_scaletrace`) 2..2	—	function
`display.graph.drawstyle`	(obj, enum `disp_drawstyle`) 2..2	—	function
`display.graph.cursor`	(obj, enum `disp_cursor`, double, double) 2..4	(double, double)	function
`display.graph.markersize`	(obj, int, int) 2..3	—	function

Lua name	arguments (min..max)	returns	exists
`display.graph.markerlastcolor`	(obj, enum, int) 2..3	—	function
`display.graph.select`	(obj, buffer, enum elem, string) 2..4	—	function
`display.graph.removeall`	(obj) 1..1	—	function
`display.generate`	(enum `disp_action`, int, int, int, int) 1..5	—	**absent**
`display.press`	(string, int, int) 1..3	—	**absent**
`display.getpixel`	(int x, int y) 2..2	int	**absent**
`display.getchecksum`	(int x1, int y1, int x2, int y2) 4..4	int	**absent**
`display.dump`	(enum `disp_dump`) 0..1	string	**absent**
`display.capture`	—	—	**absent**
`display.getobject`	(string) 1..1	object	**absent**
`display.autolocal`	(int) 1..1	mirror	**absent**

The graph family is registered **under** **`display.graph`**, not on `display` — which is why `display.scalex` reads nil while `display.graph.scalex` is a function. Two descriptors, method indices 38 and 52, carry full signatures and are registered under no name at all, so they are unreachable from Lua.

Enforced ranges recovered alongside the counts, each refused with `1130` naming its bound: `setthickness` is **1..10** inclusive, and `setfill`'s percent is **0..100**.

Three of the absent functions are worth knowing about anyway, because they would be the right tools on a model that has them. `getchecksum(x1,y1,x2,y2)` hashes a screen rectangle in **one** message, whereas `getpixel` blocks on a reply per pixel — so on an instrument with both, checksum a region and never poll pixels. `dump()` walks the object tree and returns a string. And `generate` is the one function in the module that is **not** read-only: it injects front-panel actions, where `ACTION_PRESS_FRONTREAR` switches the input terminals and `ACTION_PRESS_TRIGGER` starts the trigger model and can abort a running acquisition. It is absent here, which is the only reason that is a footnote rather than a warning.

No path in this module writes flash, EEPROM or NVRAM. Calibration is untouched by the display API.

Signatures worth noting against the documented ones: `display.create` is `2..16` arguments, `display.input.option` takes up to fourteen strings, `display.waitevent` is `0..1` and returns two or three values, and `display.clear` takes an **optional** object — `display.clear()` with no argument is legal.

Drawing fast: the graph object, measured against the pixel path

`CONFIRMED` on 1.7.17a. The question was whether an app can draw faster than the built-in graph by writing pixels. It cannot, and the graph is available to an app anyway.

`display.OBJ_GRAPH` is 18, and an app can create one on its own screen:

```lua
local scr = display.create(display.ROOT, display.OBJ_SCREEN, 'mygraph')
local g   = display.create(scr, display.OBJ_GRAPH, 0, 0, 780, 390)   -- exactly 6
        parameters
local b   = buffer.make(200, buffer.STYLE_WRITABLE)
buffer.write.format(b, buffer.UNIT_VOLT, buffer.DIGITS_6_5)
for i = 1, 200 do buffer.write.reading(b, math.sin(i / 12) * 5) end
display.graph.add(g, b)
display.changescreen(scr)
```

Measured, with the event log clean throughout. `dmm6500-fft/docs/PERFORMANCE.md` holds the cost model the per-call figures come from:

step	time	note
`buffer.write.reading`	**65 µs** each	256 writes, function cached in a local; 72 µs through the table
`buffer.clear()`	0.2 ms	
`display.graph.add(g, b)`	**4 ms**	one call, any number of points — a task round trip, see "The graph"
`display.graph.removeall(g)`	**0.24 ms** idle, 10-60 ms otherwise	it waits for the display task's pending paint of the previous frame
256 points, refill and one repaint	**~20 ms**	the graph phase of a 12.5 fps frame
1,600 `setpixel` writes	0.020 s	12.7 µs each
`display.flush`	**0.203 s**	an 800x480 surface both times — see "Flush is a bulk copy"

An earlier revision of this table gave 140 µs per write, from 0.028 s for the 200-iteration loop above, `math.sin` and uncached lookups included. The direct measurement supersedes it.

So the pixel path is capped at about 5 Hz by a flush that copies the whole surface, while the graph path refills and repaints 256 points in about 20 ms and brings axes, auto-scaling, grid, labels, line and marker styles and legends with it. Nothing an app writes per pixel competes with the firmware's own trace renderer, which runs in the display task rather than in Lua.

Three things the create call will tell you if you get them wrong, each in its own event: `1714` "Object type Graph must have 6 parameters", `1130` "Parameter width, expected value from **120 to 799**" (so 800 is one too wide), and for `graph.add`, `1127` whose text is "reading buffer or channel list" -- the argument must be a reading buffer or a channel-list string.

A writable buffer is how an app plots arbitrary numbers. `buffer.make(n, buffer.STYLE_WRITABLE)` then `buffer.write.format(b, unit, digits)` then `buffer.write.reading(b, value)`. A standard-style buffer refuses the writes with event `4926`, "must be of a 'writable' style to use this function", and a plain Lua table is refused with `1138` — there is no bulk route into a buffer, so a refill costs 65 µs a point. The default x axis is the buffer's **timestamps**, and points written faster than the timestamp resolution bunch together. A numeric x axis is the x-y form of `add`, which is the path the frequency crash most likely took — see "The frequency crash" below.

display.setpixel writes the framebuffer directly

`CODE`. It is the one function in the module that does **not** post a message to the display task. The whole call ends in a single store:

```
((u32 *)(surface + 0x24))[x + stride * y] = (alpha << 24) | (colour & 0xFFFFFF);
```

```
display.setpixel(id, x, y, colour)          -- alpha defaults to 255
display.setpixel(id, x, y, colour, alpha)   -- alpha is the LOW BYTE of argument 5
```

So the pixel format is `0xAARRGGBB`, and **the top byte of argument 4 is discarded** — packing alpha into the colour does nothing; it has to be argument 5. The first argument is an object handle, like every other `set*`, so `setpixel(x, y, colour)` is refused with `-109`.

Since there is no message and no round trip, the cost of drawing this way is the Lua-to-C call itself — about 4 µs measured. That is a few milliseconds for a trace and over a second for a screenful, which is consistent with this project's own finding that `setpixel` was too slow to draw a spectrum with while the graph widget was not.

The coordinate range is per object, not 800x480. It comes from the object's own width and height, and an out-of-range coordinate posts `1130` quoting the real bound — so a deliberate out-of-range call is a way to ask an object how big it is. The object must also advertise a pixel capability or the call does nothing; `OBJ_IMAGE` is the likely candidate and which types qualify is unread.

setpixel has a bulk form that resamples an array over a whole object

`CODE` for the path and the resampling, `partial` for the Lua shape of the argument — and it is the most promising thing in this document. If argument 4 is an **aggregate** rather than a number, `setpixel` takes a different path: arguments 2 and 3 become a source **width and height**, argument 4 is read as a `srcW x srcH` array of doubles each holding a packed `0xRRGGBB`, and the firmware **bilinearly resamples it over the object's entire area**. Indices are clamped, so a source smaller than 2x2 is not useful.

One call would then paint an entire object with no per-pixel Lua overhead, which on a marshalled API where every other call costs microseconds is the difference between a drawable spectrum and a slideshow. What is not yet established is the exact Lua value argument 4 must be: the handler dereferences two levels before treating it as an array, which is consistent with a flattened table, but the flattener has not been read.

An out-of-range object handle may corrupt instead of failing

`HAZARD`, `partial` — stated carefully because the consequence is severe and the evidence is an inference rather than a read. Object handles are **10 bits**: `getpixel` clears those bits and the create path sets `id & 0x3FF`, both `CODE`. What follows from the field width, and has **not** been confirmed by reading a bounds check, is that a handle of 1024 would address object 0 rather than being refused — a stale or computed handle silently writing to a live object, with no event and nothing in the log. This project's pool is 463 objects, well inside the mask, so every handle it could hold aliases something real. Worth designing against; not yet worth asserting.

The graph: a trace is a copy in another task, fed by a subscription

`CODE` throughout unless marked, from a static pass over 1.7.18a; `notes/firmware/graph/GRAPH.md` has every address and the method. Nothing in this section was measured on the instrument except where it says `CONFIRMED`: those figures come from the table above and from the probe of 2026-10-23 recorded in `notes/firmware/graph/MEASURED.md`.

A display call is a message to a task

Three layers. **Same thread:** the shared trampoline marshals the arguments and calls the display module's `execute`, the 59-case switch at `0x002CF58C`. Every `display.graph.*` case first calls `disp_check_supported(id, "<cap>", 0x009B11C8)`, whose allow-list is `{18, 23, 0}`, Graph and Graph Screen. **The display message:** the case fills the shared 0x6C-byte record at `0x009E7BB0` — `+3` length, `+4` object id in the low 10 bits, `+6` opcode, payload from `+0xC` — and posts it with `q_post_msg(DAT_00cef0b8, rec)`. Then it either waits, `ev_wait(DAT_00cef0b4, 0, timeout)` with a two-second timeout (`{0, 2, 0, 0}` at `0x00670748`, `IMAGE`; `CONFIRMED` by a cursor set with the 9.9e37 "off" value, which the display task never acknowledges and which returns after 2000.1 ms), returning 0 on reply, -1 on timeout and -2 on abort, or returns at once. **The "PEG Main" task:** `relay_disp_record_to_peg` (`0x000B79EC`) maps the opcode through a 41-entry table at `0x00547AD0` (`IMAGE`: entry *n* is PEG type `0x40C8 + n`, except `0x0F` → `0x40D8`, `0x12` → `0x40D7`, `0x21` → `0x40EA`), copies the record into a per-object parameter block of stride `0x2F0`, and pushes a PegMessage. The custom-screen window's `Message()` (`0x00054510`, 18.8 KB) applies it to the widget whose pointer sits in the GUI object table (`0x00D17BE0 + id*0x260`, pointer at `+0x50`) and acknowledges with `c2c_send_text(DAT_00a75844, "S$DE")` (`0x00547A44`).

So a synchronous call's time is a task round trip — queue, context switch, the receiving task's work up to its acknowledgement, and the wake-up — plus whatever that task had queued or in progress ahead of it, and PEG paints only when its queue is empty. `CONFIRMED` on 1.7.17a: `add` takes 0.5-5 ms whether the buffer holds 16, 256 or 710 points; `removeall` takes 0.24 ms when the display task is idle and 10-60 ms when the previous frame's paint is still in progress; a cursor set is acknowledged in 0.10-0.16 ms and a second one straight after it is just as fast; `scaley` with unchanged limits is acknowledged in 1.9 ms and a second one straight after waits 54 ms, because the first triggered a full repaint. The paint of a 784x300 graph with a 256-point line trace is therefore roughly 20-50 ms of the display task's time.

Replies that carry data are text, `S$DE_<id>_<subcode>_<values…>`, parsed by `disp_rx_handler` (`0x002872C4`) and stored in the Lua-side object record, 176 bytes at `0x009D1BB0 + id*0xB0`:

subcode	record fields	event bit	meaning
`0x0F`	`+0x38 +0x40 +0x48 +0x50` xmin, xmax, ymin, ymax	2 `EVENT_SCALE`	scale changed
`0x10` - `0x12`	`+0x58/+0x60`, `+0x68/+0x70`, `+0x78/+0x80`	4 `EVENT_DRAG`	vertical, horizontal, trigger cursor dragged
`0x13` - `0x15`	the same pairs	8 `EVENT_CURSOR`	cursor placed
below `0x0F`	`+0x88 +0x90 +0x98 +0xA0`	1 `EVENT_PRESS`	touch, in data coordinates

If the record's event mask (`+6`) has the bit and a handler string is installed, `disp_dispatch_event` (`0x00286C3C`) substitutes `%value` and `%id` and queues the chunk for the interpreter; otherwise the message counts as an acknowledgement.

The trace is a copy, fed by a subscription

The display task does not read the buffer. `graph_trace_add` (`0x0002D3C4`) allocates an **82,464-byte** trace object per `add` — `0x14220` , built as `lis r26,1` / `addi r26,r26,0x4180` / `addi r3,r26,0xA0` at `0x0002D438` - `0x0002D440` , then `bl 0x0018CB1C` at `0x0002D444` — and appends its pointer to the graph widget's trace array at `graph+0x3EC` . It holds the buffer name (32 bytes at `+0x08`), the subscription handle (`+0x30`), per-trace x and y ranges with pixels-per-unit, and from `+0x220` a **720-slot ring of 56-byte records**: three floats at `+0x1C/+0x20/+0x24` (the value, and for binned data the min and max around it), flags at `+0x28` (bit 7 overflow), the x value at `+0x2C/+0x30` as seconds and nanoseconds on a time axis or two floats on a numeric one, and scratch at `+0x00` - `+0x14` where the extent passes write pixel coordinates. **Values are floats**; the display task never sees the buffer's doubles. 720 slots against a 710-pixel plot is one record per pixel column; the appender (`0x00022140`) caps the count at `0x2D0` and wraps.

The ring is filled by streaming. `graph_trace_add` calls `bufsub_subscribe` (`0x000B4708`), which sends `S$BE` (`0x005479A8`) and the buffer name to the buffer module; an empty name means `defbuffer1` . There (`0x002BB684`) the subscription installs two hooks on the buffer object itself (`rbuf_install_hooks` , `0x0027F824`): an append hook (`0x002BC8BC`) and a clear hook (`0x002BD87C`). The append hook drains the readings pending for its cursor and sends them **batched, `floor(239 / record size)` per message** (`0xEF / size` , around `0x002BCE40` ; a 0x14-byte header plus the records, with a more-follows bit). The clear hook sends one clear marker. On the display side the appender copies each record into the ring and, only if the graph's pending flag `+0x5B0` is clear, posts PEG **`0x403E`** and sets the flag — however many records arrive, one repaint is outstanding at a time. The graph widget's `Message()` (`0x0002A164`) handles `0x403E` with `graph_new_data_process` (`0x00028F94`) and, if anything changed, the repaint (`0x00033540` : axes, traces and cursors between `BeginDraw` and `EndDraw`). The extent passes walk only the records not yet processed (`+0x1F8` to `+0x200`), so the per-batch cost is O(new records) for extents plus O(ring) for the draw, whatever the buffer's size.

Three consequences. `buf.clear()` empties the ring and repaints, which is the flicker a refill in place shows (`CONFIRMED` , the spectrum analyser's own observation). The acknowledgement that ends `add` 's 4 ms is sent **before** any readings arrive; the first paint happens when the burst lands. And the initial burst is decimated: the stream request `S$BA` (`0x00527550` , sent by `graph_trace_rescale` at `0x00027EC0`) names a point budget of `0x2C6` = 710 for the full-width graph or `0x288` = 648 in the latest-window mode, and a larger buffer is decimated by the buffer module before it is sent. For 256 readings nothing is decimated.

The append hook drains a staging ring, not the buffer

`CODE` from `notes/firmware/graph/fns11.c` and `fns8.c` , with the record layout and every live address `CONFIRMED` on a DMM6500 1.7.17a by `dmm6500-fft/tools/probe_substage.py` on 2026-11-08.

The send is synchronous, and two earlier attributions were wrong. `dispatch` ends at `0x001E7240` with an indirect call through the message object's own vtable:

```
lwz   r9, 4(r27)      ; the vtable
lwz   r0, 8(r9)       ; slot 2
addi  r4, r1, 104     ; an out-parameter
mtctr r0
bctrl                 ; <- the send
lwz   r31, 64(r1)     ; a RESULT COUNT comes back
lwz   r6, 12(r11)     ; clamped to a capacity
bl    001E6A64        ; and is unmarshalled
```

So **`0x001E6A64` is reply unmarshalling, not argument marshalling**, and `0x00212D94` at `0x001E7218` sits in the error path downstream of the `1138` / `1139` / `1147` arity checks, which makes it event reporting rather than the send. `fwmem-DISPATCH.md` left "does the caller block?" open; it does. A fire-and-forget post has no return values to decode.

The hook never reads the reading buffer. `bufpub_append_hook` (`0x002BC8BC`) copies 28-byte records out of a ring the **subscription** owns, and the subscriptions live inside the buffer object itself, so every address below is reachable from a descriptor with no firmware constant:

offset	field
`desc + 0xF60`	lock taken around installing or removing a subscription
`desc + 0xF64`	number installed
`desc + 0xF68`	subscription slot 0, stride `0xE48`
`sub + 0x00/0x04/0x08`	the three arguments `rbuf_install_hooks` was given
`sub + 0x10`	read cursor, advanced `0x1C` a record, wraps at `+0x94` to `+0x8C`
`sub + 0x70`	lock, held for a whole drain
`sub + 0x78`	enabled; zero drains nothing
`sub + 0x84`	record size, `0x1C`
`sub + 0x8C`	ring start, `malloc(0x5780)` = 22,400 bytes = **800 records**
`sub + 0x90`	write cursor
`sub + 0x94`	ring end
`sub + 0xA0`	pending count, decremented per record drained

`rbuf_install_hooks` (`0x0027F824`) allocates that ring on first use and memsets 800 slots. The drain loop is the whole mechanism:

```
iVar14 = *(int *)(param_1 + 0x10);                    // read cursor
*(int *)(param_1 + 0xa0) += -1;                       // pending--
FUN_004ac57c(puVar9, iVar14, *(int *)(param_1 + 0x84));  // copy the record out
uVar8 = iVar14 + 0x1c;                                 // advance
if (*(uint *)(param_1 + 0x94) <= uVar8)                // past the end?
    uVar8 = *(uint *)(param_1 + 0x8c);                 //   wrap
*(uint *)(param_1 + 0x10) = uVar8;
```

batched `0xEF / 0x1C` = **8 records a message**, with a more-follows bit.

The 28-byte wire record, measured rather than read off the code. One append of `-42.5` dB at `12345.0` Hz into a live trace buffer, dumped at the write cursor before and after:

```
before  00000000 c2c4670e c2c4670e 00000000 41c35000 41c35000 40000000
after   00000000 c22a0000 c22a0000 00000000 4640e400 4640e400 40000000
```

wire	meaning
`+0x00`	zero in every record seen
`+0x04`	the reading, float32

wire	meaning
`+0x08`	the reading again -- the min/max pair, equal for an unbinned point
`+0x0C`	flags, read as a byte at `param_6[3]`; **bit `0x80` is a time axis**, clear is numeric
`+0x10`	the x value, float32 (this app's extra value, Hz)
`+0x14`	the x value again
`+0x18`	`0x40000000`, which is `2.0` as a float

The display side copies that **verbatim** into its 56-byte trace record at `+0x1C` -- `graph_fn_22140_timer` (`0x00022140`), `FUN_004ac57c(iVar2 + 0x1c, param_6, param_5)` -- with the count at `trace+0x218` capped at `0x2D0` = 720, the write cursor at `trace+0x214`, the ring bounds at `trace+0x208` / `+0x20C`, and the graph's pending-repaint flag at `graph+0x5B0`. Note the layout is one word later than the prose above this section implies: the "value" slot at `trace+0x1C` is the record's zero word, and the reading lands at `trace+0x20`.

The hook runs inline, on the appending thread. `CONFIRMED`: a single append moved the write cursor exactly 28 bytes and left `pending` at zero, so the drain had already happened before the call returned. There is therefore no concurrent reader of the staging ring, and the lock at `sub+0x70` guards against another subscriber, not against us.

Live on a DMM6500 1.7.17a, one trace bound to a 510-point buffer: descriptor `0x035BC570`, buffer lock `0x03248C20`, 1 subscription at `0x035BD4D8`, ring `0x035430F8..0x03548878` (22,400 bytes), record size 28, read and write cursors equal, pending 0, enabled 1.

What this closes. Writing readings into a buffer's element storage cannot make the graph draw, and "fill 254 points natively plus one append from Lua" cannot work either -- not as an inference from a failed experiment, but because the hook stages one record per sanctioned write and drains only what `pending` counts. A native element write puts nothing in the ring, so there is nothing to drain; one append stages one record, so exactly one point draws. That is precisely what was measured in both cases.

What it opens, and what it closes. `CONFIRMED` cost split: 76 us a write with a trace bound against 54 us unbound, so the hook is ~22 us and the IPC round trip ~54 us. Bypassing the hook alone wins under a third of the fill; the round trip is the prize.

Pre-loading `pending` does not work. `CONFIRMED`: eight records staged into the ring by hand with `+0x90` advanced and `+0xA0` set to 8 read back correctly, and the first sanctioned append then drained exactly ONE record and left pending at zero. The push path **sets** pending rather than incrementing it, so a count placed there is overwritten. The batching bound of `0xEF / 0x1C` = 8 records a message is therefore reached only by whatever stages many at once -- the subscribe-time burst -- and not by an ordinary append.

HAZARD. Advancing `+0x90` and not restoring it leaves the read and write cursors permanently out of step. The subscription's accounting then stops adding up and the service task raises **2350, "SPLT command message lost"**, as a modal dialog that reappears on OK and blocks the control socket and the LXI web panel together. `ABORT`, dead-socket termination on port 5030 and `ki.reboot` all fail, because none of them can reach the interpreter. Only a power cycle clears it. Restore both fields before anything else that can fail.

The ring accumulates across frames, so the pass-to-pass join is drawn. `CONFIRMED`, and it took three builds to get right. The trace object's ring is 720 records and the appender wraps when it fills, so the plot shows roughly the last 720 points however they arrived -- about 2.8 passes at 255 points each. A pass that

always runs `flo -> fhi` therefore leaves a `fhi -> flo` step at every frame boundary inside that ring, and the widget draws it: a line from the start of the trace to its end, on alternate frames as the wrap moves. **The trace is not closed on itself** -- an unconditional closure would draw every frame, and `display.graph.setfill(g, 0)` does not affect it either.

What removes it is making each pass END where the next BEGINS. A fold does that by running up the band and back down, ending at `flo` where the next pass starts, at twice the appends. **Alternating the direction of each pass gets the same property at one lap** -- ascending ends at `fhi` where the next descending pass begins, descending ends at `flo` where the next ascending pass begins -- and a descending pass traverses the same curve backwards, so the picture and the trace extent are unchanged. Measured on a DMM6500 at a 50 kHz span: **14 fps at 2048 against the fold's 9.5**, 17 against 11 at 1024, and no line. One or two frames glitch at launch while the ring fills.

Do not test any of this with a straight-line spectrum. A ramp's pass-to-pass join is collinear with the ramp, so it is invisible and an always-ascending pass looks clean when it is not. Two identical ascending ramps written into one ring drew a single clean line, which was read as "no return edge is drawn" -- and two builds shipped on the strength of it. Use a shape whose two ends sit at different values.

`display.setvalue` is not a bulk path for a graph. `CONFIRMED`: command 36 refuses every table shape against a graph object with event **1717**, and `display.getvalue` on the same object answers nil and 1717 too. An empty table, one number, two numbers, an x-y pair, ten values, ten `{x,y}` rows and 255 values were all refused, and `setvalue(g, t, display.ELEMENT_DATA)` gets **1138** instead. The allow-list for that call does not include a graph; it belongs to editable widgets.

How this compares with the bulk `setpixel` lead. Exactly 5 of 1376 registered functions take a table: `buffer.channelmath` (232), `buffer.write.reading` (226), `display.graph.add` (39), `display.setpixel` (30) and `display.setvalue` (36). `setpixel`'s table form is the resampling path two sections above, and it is a different kind of answer from this one: it paints an object in one call with no round trip at all, but it bilinearly resamples a source array over the object's **whole area**, so using it for a trace means rasterising the plot into a `srcW x srcH` table of doubles every frame and giving up the graph widget -- and with it the axis, cursors and autoscaling that widget draws. Building a table that size in Lua costs more than the 510 appends it would replace, and anything drawn over the graph is repainted by it, so the widget would have to go entirely. The staging ring keeps the widget and the axis, which is the binding constraint on this app.

`display.setvalue` (36) is the one of the five that has **not** been tried or read. Its table is mandatory -- `min_args` is 2 -- and it is undocumented. Both it and `setpixel` appear under a `display.graph.` prefix in `registry.json` that is an artifact of how the sub-table is walked; the app calls the same family as plain `display.settext` and `display.create`.

Why `removeall` + `add` every frame costs what it does

The pair is the only API-level way to get exactly one repaint per frame, and it does more than it looks. `removeall` unsubscribes (`S$BB` , `graph_trace_remove_first` at `0x0002C9C4`) and **frees the 82 KB trace object**; `add` **allocates a new one**, runs eight sub-object constructors, re-subscribes, and the display task requests the buffer's current contents in one burst. Two blocking round trips plus an 82 KB malloc and free per frame, on an instrument whose heap the operator already watches. There is **no begin/end-update, batch or explicit-repaint opcode** among the module's 59 cases; the complete list is below. `drawstyle` does end in an invalidate, but the trace already repaints when data arrives, so an extra one only adds a frame.

`CONFIRMED` on 1.7.17a, 256-point frames, event log clean throughout:

per frame	Lua side	display task
refill with the trace bound	76 µs a write against 54 unbound: the append hook costs **22 µs a reading**	one repaint per coalesced batch — the partial-trace flicker
`removeall`, refill, `add`	0.24 + 14 + 1.2 ms	~22 ms for the burst and one paint, paid inside the *next* `removeall`
refill bound, then `removeall` + `add` (the spectrum analyser's order)	19.7 + ~22 + 2 ms	the same, plus the streamed batches
hide, refill bound, show	0.030 + 19.7 + 0.048 ms	**36.5 ms** of message work for 64 batches of four readings plus hide and show, then the paint

So the expensive half of the pair is `removeall`, and only because it waits for the paint the display task still owes from the previous frame; with that paint done it costs 0.24 ms. Hiding the graph (`setstate` INVISIBLE / ENABLE, both fire-and-forget as read) makes the Lua side cheap but gives the display task more to do, not less, because the subscription forwards readings four to a message where the burst sends them all at once — and since that task runs only when Lua blocks, a frame built on hide/show has to yield for those 36 ms explicitly or fall behind on every frame. The pair's merits are one burst and natural pacing; hide/show's are no blank interval and no 82 KB churn. Screenshots during 12 s loops caught a blank graph in both pair orders and none under hide/show, but only 3-4 grabs landed per loop, so that part is indicative, not measured.

The complete `display.graph.*` surface

Ten functions, registered into the `graph` sub-table at `0x002CCF78` and `0x002CCFDC` - `0x002CD2FC`. All take a graph object, else `1717`. `cmd` is the switch case, `op` the display-message opcode, `PEG` the message type on the display side; S waits for the acknowledgement, A returns at once.

Lua	cmd	body	descriptor	args	op	PEG	S/A
`graph.add`	39	`0x002D774C`	`0x00EB2430`	1..7	`0x18`	`0x40E0`	S
`graph.markerlastcolor`	40	`0x002D7938`	`0x00EB243C`	2..3	`0x26`	`0x40EF`	A
`graph.markersize`	41	`0x002D79C8`	`0x00EB2448`	2..3	`0x25`	`0x40EE`	A
`graph.removeall`	42	`0x002D7AAC`	`0x00EB2454`	1	`0x19`	`0x40E1`	S
`graph.select`	43	`0x002D7AD0`	`0x00EB2460`	2..4	`0x1A`	`0x40E2`	S
`graph.scaletrace`	44	`0x002D7B70`	`0x00EB249C`	2	`0x28`	`0x40F1`	A
`graph.scalex`	45	`0x002D7C10`	`0x00EB246C`	1..5	`0x1B`	`0x40E3`	S; one argument sends nothing
`graph.scaley`	46	`0x002D7C10`	`0x00EB2478`	1..5	`0x1C`	`0x40E4`	S; one argument sends nothing
`graph.cursor`	47	`0x002D7EB0`	`0x00EB2490`	2..4	`0x1E`	`0x40E6`	S; two arguments send nothing
`graph.drawstyle`	48	`0x002D7E30`	`0x00EB2484`	2	`0x1D`	`0x40E5`	A

Existence of all ten is `CONFIRMED` by `type()` (the table under `display.graph` above). `display.graph.delete` (cmd 49, `(string)`) is `display.delete`'s by-name form, not a graph method. `display.setevent` posts opcode 6 and `display.setstate` opcode `0x0E`, both without waiting.

`add(g [, ybuf [, yelem [, x [, xelem [, ?, legend]]]]])`. Argument 4 is declared `value` (`0x0F`) and resolved at run time; an earlier pass rendered the type word as `table`, which it is not. The body at `0x002D774C` accepts:

- `add(g)` — no buffer. The subscription's empty name defaults to `defbuffer1`.
- `add(g, ybuf [, yelem])` — y from `ybuf`, x is time.
- `add(g, ybuf, yelem, xbuf [, xelem])` with a **reading buffer** fourth: the numeric x-y plot. Without `xelem` it is forced to 2, `ELEMENT_EXTRA`.
- `add(g, ybuf, yelem, chanlist [, xbuf, xelem, legend])` with a **string** fourth: a channel list (`"101"`, `"all"`) selecting one channel of a scan buffer; only 4 or 7 arguments are accepted on this path. Anything else fourth posts `1127` "expected reading buffer or channel list", the text this bench has already seen.

What `graph_add_trace` (`0x0028D7E0`) then enforces: `yelem == ELEMENT_TIME` → `1730` "y axis cannot be time"; the x buffer must be the **same** buffer as the y buffer (`strcmp` of the two names, else `1724`); the first trace fixes the graph's kind — `xelem == ELEMENT_TIME` is a time axis, anything else numeric — and a later trace of the other kind gets `1723`. Limits: 20 traces per graph (`1722`), 30 across all graphs (`1736`), 10 graph objects (`1736`, in `disp_create_obj`). The message carries the two buffer *names*, 32 bytes each at `+0x0C` and `+0x2C`, `(yelem << 4) | xelem` at `+0x4C`, the buffers' style codes at `+0x4F`, and 16-bit handles for the channel list and legend at `+0x60/+0x62`. So `add(g, buf, ELEMENT_DATA, buf, ELEMENT_TIME)` — the spectrum analyser's call — is the time plot, and `add(g, buf, ELEMENT_DATA, buf, ELEMENT_EXTRA)` the x-y plot.

`scalex(g [, method [, axis [, min [, max]]]])` and `scaley` share one body; `cmd == 46` selects y.

- **One argument is a free read, and only works after `setevent`.** If the object's event mask (`+6`) is non-zero it returns the two doubles the display task last reported (`+0x38/+0x40` for x, `+0x48/+0x50` for y) without sending anything; if the mask is zero it returns nothing. The mask is set by `display.setevent(g, display.EVENT_SCALE, 1)`, which also makes the display task send a subcode `0x0F` report whenever the scale changes. So: enable `EVENT_SCALE` once, then poll `scalex(g)`. `CONFIRMED` on 1.7.17a with `display.ON` as the third argument: `nil, nil` before the `setevent`, `0, 0` after it until the first scale change, then the autoscaled limits (`x 0, 14.025` and `y −3.2, 3.2` after re-adding a trace of three times the amplitude).
- **Two to five arguments** go through `graph_scale` (`0x0028E150`) and wait. `axis` is `AXIS_NORMAL` 0 or `AXIS_LOG` 1; non-zero takes the limits as absolute values, and the log flag is honoured for y only (`graph+0x3A0`). `hi < lo` is swapped; `lo == hi` with two limits → `1130` "minimum and maximum values cannot be equal"; a numeric limit outside ±1e15 → `1131`; a time limit outside 0..1e8 s → `1131`. The call returns the two values it sent, not the display task's result, and an unsupplied limit is 0.0.

`cursor(g, type [, pos1 [, pos2]])`. `CURSOR_VERTICAL` 1, `CURSOR_HORIZONTAL` 2, `CURSOR_TRIGGER` 3 (`IMAGE`). Two arguments read the stored positions without a message; three or four sort the pair, store it, send it and wait. A position at or above 9e37 switches that cursor off — and `CONFIRMED`, that call is never acknowledged: it returns after the 2 s timeout, with no event. In-range positions, and off-plot ones such as 1e6, are acknowledged in 0.10-0.16 ms with no paint queued behind them, which makes an off-plot cursor set the cheapest synchronous probe of the display task's backlog. Dragging a cursor on the glass sends subcodes `0x10`-`0x15` back.

`select(g, buf [, elem [, chanlist]])` finds the trace whose name, element and channel-list handle match (`0x0002DDF4`) and makes it active: the one autoscaled in per-trace modes, the one `scaletrace` copies, the one cursors read. Selecting fires the scale and cursor callbacks and rescales every trace.

`scaletrace(g, mode)` needs a custom object id (at least `0x31` , else `1130` "need custom screen id not a system one"). `TRACE_ALL` 1 forces both scale methods off, copies the active trace's extents into the graph's range and rescales every trace to it; `TRACE_ACTIVE` 0 restores per-trace scaling.

`drawstyle(g, style)` , **`markersize(g, size [, lastsize])`** , **`markerlastcolor(g, which [, colour])`.** `size` must be 1..5 (`1130` "marker size 1 to 5") and `lastsize` defaults to it. `which` is `MARKER_LAST_TRACE` 0 or `MARKER_LAST_CUSTOM` 1; `colour` is `0xRRGGBB` , default -1 for unset. All three return without waiting.

`display.setevent(g, event, enable_or_handler)` is not a graph member but is the only way to make the display task report scale changes (`0x0028B384`). `EVENT_SCALE` 2 and `EVENT_CURSOR` 8 are valid only on a graph; `EVENT_DRAG` 4 on a graph or a slider; `EVENT_PRESS` 1 on anything; `ENDAPP` , `SHOW` , `HIDE` and two values with no constant, `0x80` (knob rotate) and `0x100` (knob press), only on screens — `1717` otherwise. A string third argument is stored as a Lua chunk with `%value` and `%id` substituted; anything else is read as a boolean enable.

Scaling

`graph_set_scale_methods` (`0x0002CED0`) maps the Lua enum to the widget's internal code:

Lua x	internal		Lua y	internal
`XSCALE_OFF` 7	0 fixed		`YSCALE_OFF` 5	0 fixed
`XSCALE_ALL` 3	1 whole buffer, decimated to 710 points		`YSCALE_PER_TRACE` = `ALL` 1	10 continuous per-trace autoscale
`XSCALE_LATEST` 1	2 window ending at the newest reading, 648 points		`YSCALE_PER_TRACE_ONCE` 2	9 autoscale until six points exist, then hold
`XSCALE_GROUP` 2	7, with `+0x3E0` = 1		`YSCALE_SWIM` 3	0xB swim lanes
`XSCALE_AUTOBIN` 4	8 histogram auto-bin		`YSCALE_SHARED` 4	0xC one range for all traces
`XSCALE_SWIM` 5	9		`YSCALE_SMART` 0	the remembered mode, flagged smart
`XSCALE_SHARED` 6	10			
`XSCALE_SMART` 0	the remembered mode, flagged smart			

Smart mode (`0x00027950`) on a time axis picks `LATEST` when the data span exceeds the visible span by a 0.7 ratio, `ALL` with at most four records, `GROUP` for a scan group, and a fixed window otherwise; the remembered y mode is `SHARED` with more than one trace and 9 with one. The first trace's kind sets the defaults (`graph_set_xaxis_kind` , `0x0002BF14`): a time axis gets x internal 4 with the smart flag, y `0xC` , draw style `BOTH` , ranges 0..10 s and 0..10; a numeric axis gets x `ALL` , y 10, style 9, both ranges 0..1 then re-ticked.

Autoscale has a 35 % hysteresis band. `trace_y_extent_b` (`0x00025E70`) finds the min and max of the new records (in log10 for a log axis, floored at 1e-35) and changes the trace's range only if the new min lies outside `[lo, lo + 0.35·span]` or the new max outside `[hi − 0.35·span, hi]` (0.35 at `0x00527510` ,

`IMAGE`) — which is what stops a live trace re-ranging on every reading. `graph_fn_2705c` (`0x0002705C`) then combines traces: `SHARED` gives all traces one range rounded by `nice_range` (`0x00026590`); **swim** places trace *i* of *n* in its own vertical band of width `1.1·(max − min)`, stacked so the lanes do not overlap; a trace with no data inherits the active trace's range; a histogram floor is clamped to 0; trigger-cursor mode pads 10 % around the cursor level. Time-axis x autoscale is done by the stream request (`S$BA` with the window and point budget), with 0.85 / 0.2 / 1.2 factors (`0x005274E0`-`F0`) deciding when the `LATEST` window slides; numeric x axes autoscale like y, with the same band.

Ticks. `axis_tick_engine` (`0x0002338C`): the step is a nice number — a floor-log10 picks the decade, `ceil(step / decade)` snaps up, and one more decade step is added if the tick count exceeds the axis maximum; six divisions by default (`0x005274C8`). The label exponent is the engineering exponent of the larger limit, rounded to a multiple of 3, and labels switch to an offset-plus-delta form when the span is more than three decades below the magnitude. Time axes (`time_axis_ticks`, `0x0000C1C4`) choose seconds, minutes or hours formats with the switch at 60 s and the day at 86,400 s — the reason the spectrum analyser keeps its scaled "seconds" below 59. `HAZARD`: the tick loop `for (t = lo + first; t < hi + step/2; t += step)` has no count limit other than the step; the display task's own guard is `1731` "Graph scale values vary too greatly in accuracy" when `log10(|lo|) − log10(|span/10|) > 9`, and limits that pass the Lua-side range checks but fail it are refused with the previous scale kept.

Touch is handled in the same widget: a one-finger drag of more than 10 pixels switches the x method off and pans; two fingers zoom, with span limits 1e-7..1e8 s for time and 1e-11..1e19 for numbers.

Draw styles, and why there is no histogram

`drawstyle` writes the widget's style mask at `graph+0x59C` (`graph_set_drawstyle`, `0x0002D37C`): `STYLE_LINE` → 1, `STYLE_MARKER` → `0xC`, `STYLE_BOTH` → `0xD`; on a numeric-x graph bits 1-2 are cleared, giving 1 / 8 / 9. Bit 0 connects points, bit 2 draws markers, bit 3 is the histogram and x-y point form. The painters select on the graph's axis kind at `+0x1DC`: 0 time, 1 time with delta records, 2 numeric x-y, 3 histogram.

Histogram mode is not reachable through `display.graph.add`. The add handler calls `graph_set_xaxis_kind` with 0 or 2 only; kinds 1 and 3 are set by the built-in Graph and Histogram screens (`SCREEN_GRAPH` 11, `SCREEN_HISTOGRAM` 13), whose setup code at `0x00034D64`-`0x00037900` calls `graph_trace_add` directly. There is no hidden `STYLE_*` value, and `XSCALE_AUTOBIN` only acts on a kind-3 graph. `ABSENT` as an API feature. Trace colours come from the firmware's own per-trace table (`0x003185B0`, red `0xFF0000` fallback); marker size and the last-marker colour are the only other appearance controls.

Every display-message opcode

From the case bodies and the display-side dispatcher: 1 create, 2 delete, 3 setcell, 4 setcolor, 5 setabbrv, 6 setevent, 7, 8, 9 flush, `0xA` setminmax, `0xC` popup menu, `0xD` setposition, `0xE` setstate, `0xF` settext, `0x10` setthickness, `0x11`, `0x12` image data, `0x13` setvalue, `0x16`, `0x18` graph.add, `0x19` graph.removeall, `0x1A` graph.select, `0x1B` graph.scalex, `0x1C` graph.scaley, `0x1D` graph.drawstyle, `0x1E` graph.cursor, `0x1F` getpixel, `0x20` getchecksum, `0x21` dump, `0x25` graph.markersize, `0x26` graph.markerlastcolor, `0x27` popup-menu list, `0x28` graph.scaletrace. **None is a batch, a begin/end or an explicit repaint.**

Enum values settled statically

Six constants are not in the data tables; they are registered by code as `{name, r30}` / `{name, r31}` pairs, where `r30` is `li r30,0` at `0x002CC0E0` and `r31` is 1 — pinned by the `STYLE_LINE/MARKER/BOTH` table registered 60 bytes earlier as `{r30, r31, r28}` with the data-table values 0/1/2:

constant	value	registered at
`AXIS_NORMAL` / `AXIS_LOG`	0 / 1	`0x002CED98` - `0x002CEDC4`
`TRACE_ACTIVE` / `TRACE_ALL`	0 / 1	`0x002CEDD4` - `0x002CEE00`
`MARKER_LAST_TRACE` / `MARKER_LAST_CUSTOM`	0 / 1	`0x002CECA0` - `0x002CECCC`

`CONFIRMED` on 1.7.17a: `tonumber` of the six, in the order `AXIS_NORMAL`, `AXIS_LOG`, `TRACE_ALL`, `TRACE_ACTIVE`, `MARKER_LAST_TRACE`, `MARKER_LAST_CUSTOM`, reads `0 1 1 0 0 1`.

The frequency crash

What the code establishes, against the `HAZARD` recorded under `display.graph`:

- **The unit of an axis comes from the buffer, not from a `display.*` argument.** The `0x407F` handler reads the subscription's unit code and looks the symbol up through `DAT_009B034C[buffer.UNIT_*]` and `DAT_00526A44[(code+1)*4 + range]` into the symbol table at `0x00526580`. For `UNIT_HERTZ` 9 the row is `09 17 17 17` → index 9 → `"Hz"`. **The Hz path is well-formed**; the only garbage row is code 22, which no `buffer.UNIT_*` produces. The axis label is not read from the two 21-entry tables at `0x006B3290` and `0x006B4E28` where an earlier pass found Hertz as an empty string.
- **The axis kind is decided by the x element, not by the unit.** `add(g, buf, ELEMENT_DATA, buf, ELEMENT_EXTRA)` gives a numeric x axis whatever the extra element's unit; `add(g, buf, ELEMENT_DATA, buf, ELEMENT_TIME)` gives a time axis. "Setting the horizontal units to frequency" can only have meant binding x to an element whose unit was Hz — the extra element written by `buffer.write.format(buf, UNIT_VOLT, …, UNIT_HERTZ, …)` — which is the **numeric x-y path**, and no built-in screen exercises it.
- `HAZARD`: on that path the first trace arrives with extents at the sentinels ±1e99 (`0x00527500/08`) until the first records are processed; `graph_select_first_trace` copies trace extents into the graph range when the x method is 0; and the tick loop has no count bound. A sentinel range reaching the tick loop would write the tick array far past its end. Whether that ordering can occur depends on message timing that cannot be read statically.

The fault is not pinned, and the failing call was never captured. The discriminating experiment: on a `STYLE_WRITABLE_FULL` buffer with extra unit `UNIT_HERTZ`, bind `add(g, buf, ELEMENT_DATA, buf, ELEMENT_EXTRA)`; then the same with extra unit `UNIT_VOLT`. If both crash the fault is the x-y path; if only the first, the unit. Event log clean, and read the PC off any bluescreen.

bit: the undocumented bitwise table, and when it is worth calling

`CONFIRMED`. `bit` is a global table on this instrument and holds three functions. Lua 5.0.2 has no bitwise operators, so this is the only way to get one without writing a loop. Nothing in any Keithley document mentions it.

```
bit.bitand(a, b)   bit.bitor(a, b)   bit.bitxor(a, b)
```

Measured, with the event log read either side of every call and **zero events** filed:

call	result	what it establishes
`bit.bitand(240, 60)`	48	
`bit.bitor(240, 15)`	255	
`bit.bitxor(255, 15)`	240	
`bit.bitand(0xFFFFFFFF, 0x12345678)`	305419896	the full 32-bit range works
`bit.bitor(2147483648, 1)`	2147483649	results come back unsigned, not sign-wrapped
`bit.bitand(255, 60, 15)`	60	**two arguments only** — a third is silently ignored

There is no shift, rotate or complement: `lshift`, `rshift`, `arshift` and `bnot` are all nil, as are `band` / `bor` / `bxor` — the names carry the `bit` prefix even inside the table. Shifts have to be done with `* 2^n` and `math.floor`, and a complement with `bit.bitxor(x, 0xFFFFFFFF)`.

It is not automatically a speed win, and for masking it is a loss. 20,000 iterations each, loop overhead subtracted:

loop body	per iteration
`x = bit.bitand(i, 255)`	4.1 µs
`x = bit.bitxor(i, 255)`	4.1 µs
`x = math.mod(i, 256)`	3.5 µs
`x = i - math.floor(i / 256) * 256`	**2.7 µs**

A Lua-to-C call on this interpreter costs about 4 µs — roughly 1,600 cycles at 400 MHz — and that dominates the operation itself. So:

- **Masking to low bits: use arithmetic.** `i - math.floor(i/256)*256` is 1.5x faster than `bit.bitand(i, 255)`, and `math.mod` is faster too. The call overhead is the whole cost.
- **A general AND or OR of two arbitrary values: use `bit`.** The arithmetic equivalent needs a loop over bit positions, which is many times 4 µs.
- **XOR: use `bit.bitxor` always.** There is no cheap arithmetic equivalent at all.

The rest of that table is not reachable from Lua

The same internal table in the image carries `memread`, `memwrite`, `peek8`, `peek16`, `peek32`, `poke8`, `poke16`, `poke32`, `spindelay`, `trace`, `intercept`, `aborthook`, `remotecomm` and a flash protection call. **Every one of them is nil on this firmware**, under both `_G` and the `memory` table, which is itself a table with none of these members. `IMAGE` for their existence, `ABSENT` for their reachability.

That is worth stating precisely because of what they would be able to do. A poke or a flash protection call reached from a script is the one thing on this instrument that could destroy calibration data irrecoverably, and the measured answer is that Lua has no entry point to them. No argument list for any of them is recorded here, and none should be worked out: the names are listed so that nobody goes looking for them, not so that anybody can use them.

Namespaces nothing has mentioned

`IMAGE`, from the registration walk: 63 namespaces in all, against the 44 a dotted-string count suggests. Each of these holds registered constants rather than being a loose string: `serial` 24, `sysinfo` 23, `analogtrigger` 12, `interruptcontroller` 14, `interruptmapping` 7, `format` 9, `coupling` 3, `filter` 4, `rel` 3, `threshold` 2, `fan` 2, `offsetcompensation` 4, plus `mpc512x` — which names the SoC, a Freescale MPC512x with an e300 core. Six have no documentary trace of any kind: `emu`, `flog`, `c2c`, `backgrounds`, `tmodel`, `sysinfo`.

`ABSENT`, and this is the shared-codebase rule again: `sysinfo`, `emu`, `flog`, `c2c`, `tmodel` and `backgrounds` are all nil as globals on a DMM6500 at 1.7.17a. Registered in the image, not reachable here.

`interruptcontroller` and `interruptmapping` are the ones to look at next: the display channel's device at `0x90000C00` is served by an interrupt handler, and if anything on the hardware side of the display is reachable from Lua, it is reachable through those.

The event number table

`IMAGE`. **676 event codes with their message text and, for most, a detail paragraph** — `docs/firmware-events.tsv`. 565 errors, 67 warnings, 44 information. This is the single most useful thing recovered from the image for day-to-day work: every unexplained number this project has chased is in it.

12-byte records in `.data`, anchored at link `0x009B098C`:

```
struct { uint32_t codeword; const char *message; const char *detail; };
// codeword = (int16 number << 16) | (severity << 8) | flags
```

The number is the top 16 bits **read as a signed int16**, which is how the negative SCPI errors share one table with the positive Keithley events. Severity 1/2/4 matches the documented `eventlog` constants.

Validated independently of everything else here: twelve standard SCPI codes come out verbatim — `-100` Command error, `-104` Data type error, `-108` Parameter not allowed, `-109` Missing parameter, `-200` Execution error, `-222` Parameter data out of range, `-363` Input buffer overrun and others.

Every code this project has ever hit resolves, including the five provoked while writing this file:

code	message
`1127`	Parameter, %s, expected %s data type
`1130`	Parameter %s, expected value from %d to %d
`1138`	Parameter %d, expected type '%s', but found type '%s'
`1703`	Object id is invalid — *detail*: "never created or has been destroyed"
`1715`	Object type %s must have between %d and %d parameters
`1716`	Object type %s parameter %d must be type %s
`1717`	Attribute %s, does not apply to an object of type %s
`2201`	File read error
`2205`	File not found

code	message
`2351`	SPLT non-critical message lost
`2710`	Unable to initiate, no block number %d found in trigger model
`4915`	Attempting to store past the capacity of reading buffer
`4926`	Reading buffer %s must be of a 'writable' style to use this function

Two codes seen on this bench are **not** in the table and therefore come from a different mechanism: `-285` (TSP syntax error) and `5076`.

`1703`'s detail settles something in this project's own history: the stale-display-object-handle event says in as many words that the id "was never created or has been destroyed".

bit: ten functions, not three

`IMAGE` for the full set, `CONFIRMED` for the first three. Each has its own real C function rather than the shared trampoline, so the signatures come from each handler's own argument checks. They match Keithley's documented 2600-series bit library one for one, which is the independent check that the extraction is right — and no DMM6500 manual mentions `bit` at all.

function	signature	returns
`bit.bitand`, `bit.bitor`, `bit.bitxor`	(number, number)	number
`bit.test`	(value, index)	**boolean**
`bit.get`, `bit.set`, `bit.clear`, `bit.toggle`	(value, index)	number
`bit.getfield`	(value, index, width)	number
`bit.setfield`	(value, index, width, fieldValue)	number

Every argument is type-checked and raises `-104` "Data type error" on a non-number; the index and width family raises `-222` "Parameter data out of range". `bitand` / `bitor` / `bitxor` carry an explicit unsigned-32 fixup, so values at or above 2^31 are handled rather than truncated — which is what the measured `bit.bitor(2147483648, 1) = 2147483649` shows. There is no `bitnot`.

`bit.getfield` and `bit.setfield` are the interesting pair for this project: extracting a bit field from a decoded byte is one call instead of a divide, a floor and a modulo.

Why the memory family is unreachable, mechanically

`memread`, `memwrite`, `peek8/16/32`, `poke8/16/32`, `trace`, `intercept`, `aborthook`, `remotecomm`, the flash-protection call and `pseudocal` are all registered by **one** function whose registrar has **zero callers anywhere in `.text`**. Not merely absent from `_G` — unreachable by construction, because nothing ever invokes the code that would publish it. That is a stronger statement than the `nil` measurements and it explains them.

Two names that both the `cal` and `nvram` token rules miss, and which are **not** protected by being dead code: `sysinfo.nvmeminit` and `sysinfo.nvmemdump`, ordinary registrations in a live registrar. `nvmem` is now in the withheld set. No argument lists for either are recorded anywhere.

Flush is a bulk copy, and the image surface is owned by its name

`CODE` and `CONFIRMED`, and the two together resolve a contradiction in my own measurements.

`display.flush(id)` is not a small message. It copies the image object's **entire pixel block** to the display task over channel 9 — the in-process channel described under "The machine the API is talking to", whose maximum message is 240 bytes — in **240-byte chunks** under a per-channel lock. So its cost is proportional to the surface's **area**: an 800x480 32-bit surface is 1,536,000 bytes, 6,400 chunks. At 0.203 s that is 7.5 MB/s, slow for a copy within one core, so the receiving task's repaint may be the real cost; a 100x100 surface against an 800x480 one would tell, and has not been timed.

I measured 0.203 s for a 400x200 object and 0.203 s for an 800x480 one, which looks like a fixed cost and contradicts that — until the second finding explains it. **The string argument to `display.create(..., OBJ_IMAGE, x, y, name, w, h)` names a surface, and the surface is what owns the pixels.** A 400x200 object created with a name already bound to an 800x480 surface displays and flushes that 800x480 surface; the `w` / `h` arguments did not resize it. Both of my flushes were copying the same surface, so both cost the same. The static read stands.

A name that has no surface gets a placeholder. Creating `OBJ_IMAGE` with a fresh name renders a small blue "?" icon, and `setpixel` into it writes nothing and posts **no event at all** — measured with 60,000 calls. On a freshly booted instrument the first fresh name allocated a surface and worked; later fresh names got the placeholder, which suggests the surface is not released when the object is deleted. So an app should create one image at startup, keep it, and size it to the region it actually redraws.

The capability allow-list for `setpixel`, for its bulk form, and for `flush` each contain **exactly one** object type: 5, `OBJ_IMAGE`. Nothing else has a pixel surface, and `OBJ_RECT` posts `1717` per call rather than failing silently.

The stale handle hazard, corrected

`CODE`. The earlier claim that a handle of 1024 aliases object 0 is **retracted**: the capability check does bounds-check, and a handle at or above 512 is refused with `1130` quoting 0..511.

The real hazard is quieter and worse. A handle in 0..511 whose **slot has been reused** passes every check, because there is no generation counter and no ownership test — the record only has to be non-zero and of an accepted type. The call then edits a live, unrelated object and reports success. With 463 objects against 512 slots, reuse is the normal case, and for the narrow allow-lists above a stale handle corrupts some *other* image. Null a handle at the point of delete.

A buffer's descriptor on a model you have never run on

Native code that reads a reading buffer in place needs the column descriptor: element size at `+0x08`, elements per chunk at `+0x0C`, chunk count at `+0x10`, the chunk vector at `+0x44`, element 0 at `+0x48`, and two owner self-pointers at `+0x54` and `+0x64` that identify it. Nothing outside the descriptor points at it — a sweep for its address found four references, three inside it and one in allocator bookkeeping — so it is found by searching for element 0's address as a big-endian word and confirming the self-pointers.

That search needs a window, and the window is the part that does not travel. A DMM7510 1.7.16 confirmed element 0 at `0x0158D7C0` and found no descriptor in `tostring({}) ± 24 MB` or `± 128 MB` around a Lua anchor of `0x010CDF10`. Its heap runs to `0x0F8A2000`, so those windows covered 143 MB of 248 MB. `tostring({})` anchors the **Lua** allocator; descriptors come from the **C++** one, and on a 6500 the two happen to sit within 11.5 MB of each other.

The buffer carries the right anchor itself:

```
getmetatable(buf).CClassRef   ->  "userdata: 353b010"     the Udata payload
peek32(0x0353B010)            ->  0x0353E330              the C++ instance
```

`getmetatable` is unprotected and `tostring` on a userdata gives the payload pointer already — **do not add 16**, that form belongs to the metatable hash walk and dereferencing it faults. The instance measured 19 KB from the descriptor, and its header lists per-column pointers that every capture has put within a couple of hundred KB of it. Sweep `± 256 KB` around those first.

The layout itself does travel. `notes/firmware/structmatch.py` indexes every `stw rS, d(rA)` whose displacement is one of the four copies of `n` — `0x78`, `0x80`, `0x84`, `0x88` — keeps the sites where all four share a base register that is not `r1`, then searches the other image for those bytes and grows the identical run. The descriptor initialiser comes back byte-identical in all ten images on hand: 6500 1.7.08 through 1.7.18 and 7510 1.7.10 through 1.7.18, 1044 bytes of it at 6500 `0x003F29A8` and 7510 `0x00221BDC`. So a descriptor a sweep cannot find is in the wrong place, not of the wrong shape — which is worth settling before moving a field offset.

Still open

Three items from the last revision are closed and have been removed: every display signature is in the table above; the event table is `docs/firmware-events.tsv`; and `display.flush` does not batch drawing — there is no batch, begin/end or repaint opcode among the 59 cases, so `flush` is the image copy and nothing else.

Settled on 1.7.17a since the last revision, each a `CONFIRMED` tag above and a line in `notes/firmware/graph/MEASURED.md`: the six enum values; the 2 s acknowledgement timeout; the one-argument `scalex` / `scaley` read-back; the 22 µs streaming cost per reading; `add` at 0.5-5 ms independent of point count and `removeall` as the call that waits for the pending paint; hide and show as fire-and-forget posts (0.030 and 0.048 ms) with 36.5 ms of display-task work behind a hidden 256-point refill; the display task running only when Lua blocks; and `add(g)` with no buffer binding the empty `defbuffer1` without an event.

Still open, each with the measurement that settles it. None touches calibration.

- **Does hiding the graph erase its area on the glass?** Hide for eight seconds and photograph the panel through the LXI grab, which takes about three seconds under load; a three-second hide was tried and both grabs landed after the show. If the old trace stays visible while hidden, a hide/refill/show frame has no blank interval at all.
- **Blank intervals in the pair orders, properly.** Only 3-4 screenshots landed per 12 s loop, because the web server shares the starved core. A loop that yields longer between frames lets more land, at the cost of representativeness.
- **The display task's paint time against graph size.** 20-50 ms for a 784x300 graph with a 256-point line trace. Repeat the probes on a 400x150 graph: if it halves, area is the cost and a smaller plot is a frame-rate lever.
- **The frequency crash.** The failing line, then the discriminating experiment under "The frequency crash". It costs a power cycle, so it is the operator's call.
- **`add(g, buf, ELEMENT_DATA, "101")`** should filter a scan buffer to channel 101; what it does on a non-scan buffer is unread.

- **What the second processor does.** Not a frame-rate question. Time `display.flush` on a 100x100 surface against an 800x480 one, if two surfaces of different sizes can be obtained under the surface-name rule above: a fixed cost means the repaint dominates, not the transfer.
- **Which queue posts `1706`.** `2351` is `q_post_msg`'s; the bench has only ever seen `1706` before a panic, from a mechanism that is unread.

And from before:

- What consumes the `ki.display` action codes, which is the HELP key answer.
- `logic.zx` and `rom.zip`, neither opened past the header. `logic.zx` is a Lattice bitstream and the only candidate on disk for whatever sits behind the `0x90000000` window.